Results, with the charts behind them

what was measured, in which unit, and what the unit hides

Abstract

Every number here comes from a run of the shipped code in exact rational arithmetic, cross-checked against an independent reference, and every chart is generated from those numbers by `docs/figs/regenerate.py`. The losses are reported at the same length as the wins, and so is the one finding that outlasts all of them: on this machine wall-clock time cannot resolve the effects being measured, so most of what follows is counted in *work* — sub-problems, branch-and-bound nodes, simplex pivots — and not in seconds.

Contents

1 What is being compared **2**

2 The margin, and how it moves with p **2**

3 Where the margin comes from, counted **3**

4 Is the margin real, or is the baseline handicapped? **3**

5 Four improvements to the exact search **5**
5.1 Why four different units 6
5.2 The measurement lesson 6

6 Hybridisation: what paid and what did not **7**
6.1 Seeding the exact search 7
6.2 But as a generator it is the stronger of the two 8

7 The asymmetry, which is the most transferable result **8**
7.1 Two ideas that feed the same mechanism do not add 8

8 Enumerating the whole front, and what the margin tracks **9**

9 The complete efficient set, for under one doubling **10**

10 The hybridisation that pays, and the three that did not **11**
10.1 First attempt: a generator. It loses. 11
10.2 Second attempt: a walk. It pays — once it is sorted. 11

11 Solving the slice equations: the largest gain here **12**

12 A wall met four times **13**

13 Fractional criteria, where the problem lives **14**

14 Five families, and a prediction that was wrong **14**

15 Where the hybrid fails, and the line that fixes it **15**

16 How the campaign was measured **16**

17 Every verdict, including the losses **16**

18 How all of this is checked **16**
18.1 The references were audited, and nothing here moved 17

1 What is being compared

Two exact methods answer (P_E) and agree on every instance. They differ in where they keep the unexplored part of the problem, and that decides the cost:

decision space truncates D with Sylva–Crema cuts. Each cut is a disjunction, so it costs p binaries and $p+1$ big-M rows, and every sub-problem is strictly larger than the last.

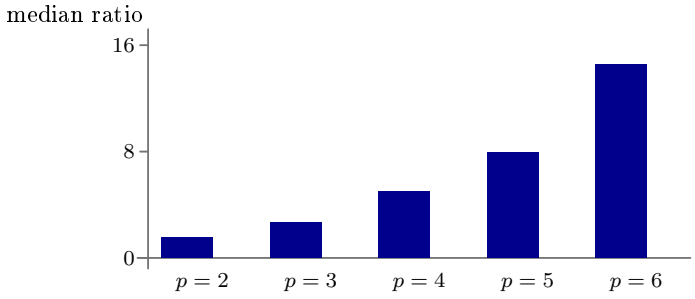
criterion space keeps a list of boxes in $\mathbb{R}^p$. The same truncation is a box subtraction; every sub-problem is D plus a few ordinary rows, and **the model never grows**.

2 The margin, and how it moves with p

Thirty instances per row, $n = 6$. The statistic is the **median of the per-instance ratios**: a ratio of totals would be decided by the single heaviest instance, which is the one thing a small sample gets most wrong.

Table 1: Decision space against criterion space, thirty instances per row.

p	decision	criterion	ratio (median)	IQR	max
2	0.07 s	0.04 s	1.52×	[1.25, 1.91]	6.0×
3	0.34 s	0.12 s	2.69×	[1.87, 3.72]	10.1×
4	0.50 s	0.13 s	4.99×	[2.39, 11.62]	76.3×
5	2.19 s	0.25 s	7.93×	[3.61, 19.98]	33.8×
6	3.67 s	0.31 s	14.57×	[7.90, 25.17]	65.7×



Three of the 150 runs were stopped by a 45 s budget — one at $p = 5$, two at $p = 6$ — and are excluded rather than counted as their budget. That censoring is *one-sided*: it can only make the decision-space method look better, so every margin here is conservative. Every instance that finished gave the same optimum on both sides.

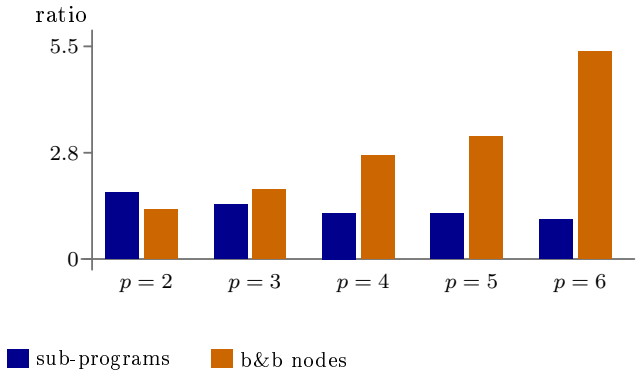
An earlier table said the curve turned over. It does not. Four instances per p gave 1.74, 2.99, 6.92, 103 and 24.3, and the text went on to explain the turn between $p = 5$ and $p = 6$. There was nothing to explain: that 103× was one instance supplying 110.95 of 112.77 seconds. With thirty instances the progression is clean and monotone. The old figures were not wrong as measurements — they are what those four instances did — but they were quoted as a trend, and four runs cannot carry one.

3 Where the margin comes from, counted

Wall-clock cannot resolve a small effect on this machine, so the campaign counts the deterministic work as well: sub-programs solved and branch & bound nodes explored, identical on every run and every machine.

Table 2: Ratios of decision space over criterion space, same thirty instances per row.

p	sub-programs	b&b nodes	time
2	1.72×	1.28×	1.52×
3	1.40×	1.79×	2.69×
4	1.19×	2.68×	4.99×
5	1.17×	3.16×	7.93×
6	**1.02**×	**5.37**×	**14.57**×



This is the whole cost model, read off the measurement. At $p = 6$ the two methods solve *the same number of sub-programs* — 1.02×, down steadily from 1.72× at $p = 2$ — and one still takes 14.57× as long. The entire margin has moved into the cost of *one* sub-problem: the branch & bound inside it explores 5.37× the nodes, because by then it carries p binaries and $p+1$ big-M rows per cut while the box search carries none. The branching that was supposed to work against the box list wins the sub-program count at small p and gives it back as p grows; the other term compounds.

4 Is the margin real, or is the baseline handicapped?

What the baseline is, and what the margin is therefore worth. The decision-space side of every comparison in this document is **our own reimplementation** of the published method, not the authors' code, and the instances are ours, not their published benchmarks. That has to be said plainly, because it changes what the numbers mean.

Both sides share the same simplex, the same exact rational arithmetic, the same branch-and-bound, the same machine and the same instances; the reimplementation follows the printed

algorithm step for step, with the two places where the printed pseudo-code is internally inconsistent corrected and the corrections documented. So the comparison *isolates the algorithmic difference*, which is arguably a cleaner question than code against code — that would also measure language, solver backend and engineering effort, none of which is the subject here.

The price of that choice is that **absolute seconds carry no information outside this implementation**. Pure Python over exact rationals with a hand-written simplex is far slower than any production solver, on both sides equally. Only the *ratios* transfer, and only the ratios are quoted.

What has *not* been done, and is not claimed: a comparison against the authors' own implementations, on their own benchmark instances. Until that is done, the correct reading of every margin here is "this algorithm beats that algorithm when both are implemented the same way", and not "this work beats those papers".

A 103× margin — and a 204× one on front enumeration — invites the suspicion that the loser was crippled rather than beaten, so the losing method was profiled instead of merely quoted.

Where the time goes. On the instance that takes 137 s to produce a 30-vector front in decision space:

Table 3: Profile of the decision-space enumeration, 137.38 s in total.

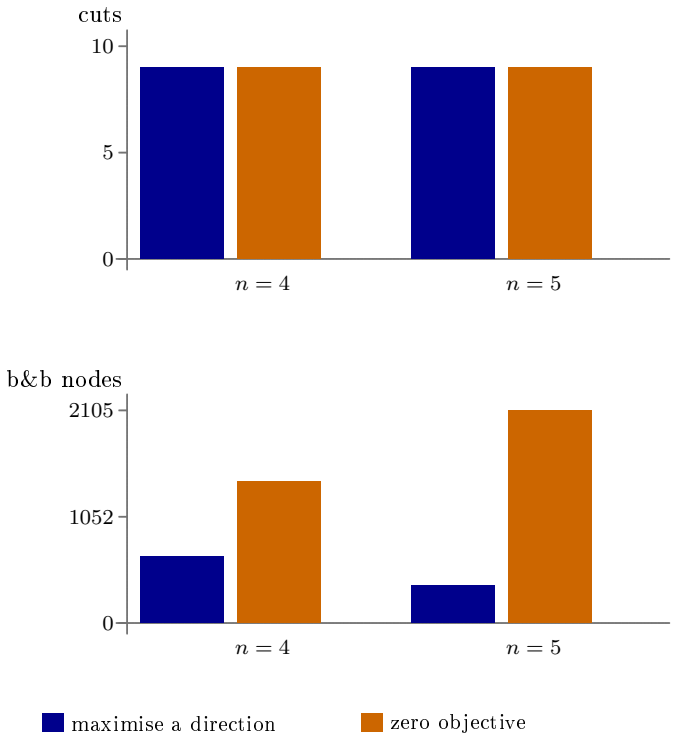
	time	share	calls
probe (the integer program)	137.21 s	**100**%	31
repair to an efficient point	0.09 s	0%	30
building and adding the cut	0.02 s	0%	30

Over those 31 calls, branch & bound explores 14983 nodes, and the model grows from 5 columns and 7 rows to 95 and 127 — exactly p binaries and $p + 1$ rows per cut. Nothing outside the sub-problems is worth optimising, and the growth of the sub-problems is precisely what the criterion-space method does not have. **The cost is structural.**

The one candidate handicap, tested. The probe only needs *some* feasible point of the truncated region, yet it maximises a linear direction. Dropping the objective looks strictly cheaper — stop at the first integer point found. Measured, it is four to thirteen times *worse*:

Table 4: Probing by maximising a direction against probing with a zero objective. Same instances, same cuts.

	cuts	b&b nodes	time
$n = 4$, maximise a direction	9	658	**1.35** s
$n = 4$, zero objective	9	1397	5.33 s
$n = 5$, maximise a direction	9	372	**0.79** s
$n = 5$, zero objective	9	2105	10.15 s



The two charts are the whole finding: the top one is flat and the bottom one is not. **The cut counts are identical** — the direction changes neither how many vectors there are nor the order they come out in — while the branch & bound inside each probe grows by a factor of two to six. A zero objective gives it nothing to prune with: every node's relaxation is worth 0, so no bound can discard a subtree until an integer point has been stumbled upon.

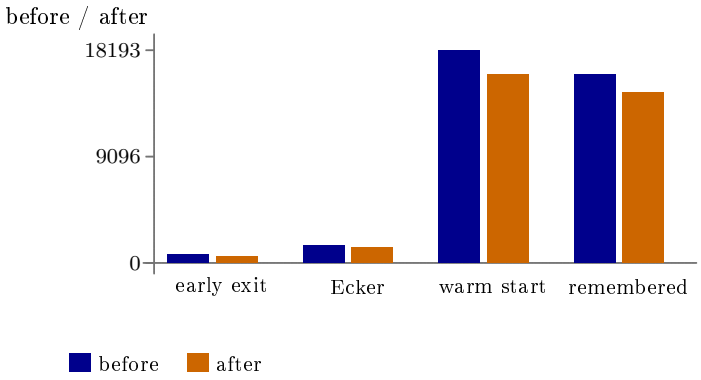
So the direction is load-bearing rather than decorative, the criterion-space method uses the same one, and the margins stand. This is reported at length because it is a plausible-looking optimisation that makes things much worse, and because it was checked while *suspecting the comparison was unfair* — which is the only time such a check is worth anything.

5 Four improvements to the exact search

Each of the four removes work without changing the answer. They are listed with the unit each was measured in, because no single unit sees all four.

Table 5: What each change removes, and what it leaves alone.

change	unit	before → after	
early exit on a hopeless bound	sub-problems	733 → 549	−25%
not re-deriving Ecker & Kouada	b&b nodes	8105 → 7618	−6.0%
warm-starting a box from its parent	simplex pivots	18193 → 16169	−11.1%
cutting on a remembered efficient point	simplex pivots	16169 → 14602	−9.7%



5.1 Why four different units

Because each change removes a different *kind* of work, and a unit that sees one is blind to the others.

- The early exit removes whole sub-problems, so it shows in a box count — but those sub-problems are the *cheapest* ones, each killed by the root relaxation the moment it sees the cutoff, so it barely moves nodes or pivots.
- Ecker & Kouada removes an integer program per inefficient box, so it shows in programs and nodes — but not in the box count, which is identical.
- The warm start removes a simplex phase I per box, so it shows only in pivots: programs, nodes and boxes are all identical.
- The remembered-point shortcut removes an efficiency test *and* changes which boxes are produced, so it is the only one of the four that is not answer-identical by construction and had to be checked against the answer instead.

5.2 The measurement lesson

The wall-clock A/B for the Ecker & Kouada change reads 0.96× — *slower* — and is faster on 9 of 18 instances. That cannot be causal: removing 125 integer programs cannot cost time.

The conclusion is about the instrument. This machine's noise floor is roughly ±30% per instance *even at per-instance minima over five repeats*, so it cannot resolve a 6% effect, and no number of repeats fixes that. Counting integer programs, nodes and pivots gives the same figure on every run and every machine.

Two habits follow, both adopted only after failing at them first:

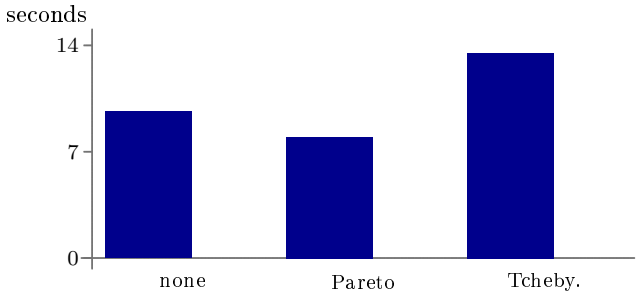
1. A measurement taken while anything else was running on the machine is **discarded, not quoted**. The first A/B for that change was contaminated exactly so and was re-run.
2. A gain predicted from the **mean** cost of a call is not a prediction. The profile showed the efficiency test at 18.4 ms per call, the dearest operation in the search — but what the shortcut removes is the repair's test at 7.68 ms, the cheap kind. Reasoning from the wrong row of the right profile overstated the gain by half.

Table 6: Seeding the box search, 18 instances. "Exactly optimal" counts the instances where the seed already attains Φ^*.

seed	total	seed cost	exactly optimal
none	9.64 s	—	—
Pareto local search	**7.97** s (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18

6 Hybridisation: what paid and what did not

6.1 Seeding the exact search



The Tchebychev seed is *slower than not seeding at all*, and both halves of that are structural. Its cost is $p + 1$ integer programs on an enlarged model — not the reference point's fault, since computing z^* took 0.01–0.13 s of the seed against 0.13–0.85 s for the scalarisations. Its quality is the real finding: the program steers by criterion-space geometry relative to z^* and **never looks at Φ**, while the local search is guided by Φ at every move.

Thirty paired instances, and one verdict softens. Eighteen instances quoted as a total has the same flaw as the margin table above, so the campaign re-ran it paired: each of thirty instances at $n = 6$, $p = 4$ is solved three times, and the statistic is the median of the per-instance speed-ups.

Table 7: What a seed is worth, thirty paired instances. The gap is $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$, zero exactly when the seed is optimal.

seed	speed-up (median)	IQR	seed gap	exactly optimal
Pareto archive	1.10×	[1.02, 1.27]	0.000	24/30
Tchebychev	1.02×	[1.00, 1.09]	0.170	8/30

The *ordering* is what every sample has said and is the part that holds: the Pareto seed is worth something, the Tchebychev seed is not. Two things change, and both are corrections.

The Pareto seed is worth **less** than the 1.21× above: a median of 1.10×, with a first quartile of 1.02×, so half the instances gain three percent or less. The win is real — both quartiles sit above 1 and the seed is exactly optimal on 24 of 30 — but it is small, and the worst instance runs at 0.618×, a genuine loss, because the verification is paid whether or not the bound can use it.

And the Tchebychev seed **does not lose here; it does nothing**: not 0.71× but 1.02×, IQR [1.00, 1.09] — a wash. The verdict that survives both samples is the weaker one, *it never pays*. What is stable is the quality figure that explains it: a median gap of 0.170 against 0.000, and 8 of 30 exactly optimal against 24 of 30.

6.2 But as a generator it is the stronger of the two

A weighted sum maximises only at *supported* efficient points. Varying the Tchebychev weights reaches unsupported ones too, which is what the literature uses it for.

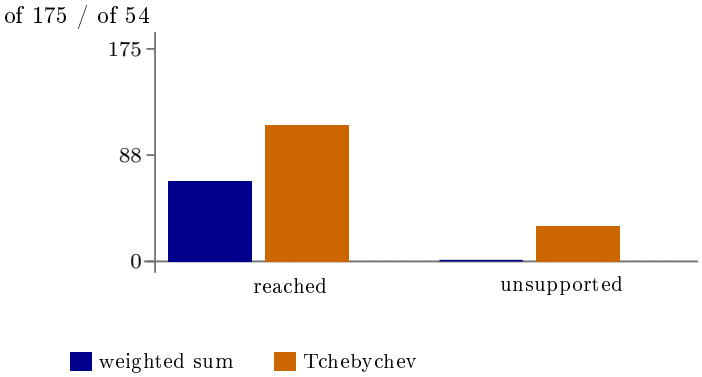


Table 8: Reach from the same grid of 27 weight vectors, 20 random instances.

	efficient points reached	of which unsupported
weighted sum	66/175	**0/54**
augmented Tchebychev	112/175	29/54

All 540 optima returned were efficient. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, and 1080 optima with none inefficient.

7 The asymmetry, which is the most transferable result

Table 9: The *same* free optimum, handed to each exact method at iteration 1.

exact method	effect	mechanism
criterion-space box search	1.42×, up to 52% fewer boxes	boxes discarded whole
decision-space cut method	**zero** iterations saved	none

The heuristic is identical in both rows. What differs is whether the exact method has a mechanism through which a lower bound can remove work: a box is compared against the incumbent *before* it is split, and failing that comparison deletes it with every descendant it would have had; what the decision-space method cuts is decided by the efficiency test on its own maximiser, and Φ_{opt} enters only as a cutoff inside a branch and bound that would have found the same point anyway.

So "hybridise an exact method with a metaheuristic" is not one question with one answer, and the mechanism is cheaper to check than the hybrid is to build and measure.

7.1 Two ideas that feed the same mechanism do not add

Either alone removes about a quarter; together they remove about a quarter. On top of the seed the exit is worth 4% rather than 25%, because a good incumbent arrives early enough that the hopeless tail barely forms. Two ideas add only when they feed different mechanisms; two that feed the same one are one idea, measured twice.

Table 10: Sub-problems solved over the 18 instances.

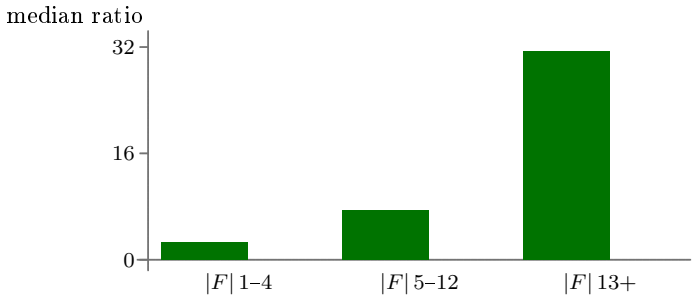
	boxes solved	reduction
neither	733	—
early exit only	549	25%
metaheuristic seed only	562	23%
both	539	26%

8 Enumerating the whole front, and what the margin tracks

The same substitution works on complete enumeration of the non-dominated set, and there the mechanism is visible directly: **every vector is a cut**, so in decision space a larger front means more binaries and more big-M rows on every sub-problem that follows, while the box list never grows. The campaign swept the constraint tightness on purpose to move $|F|$ — fifteen instances at each of three tightnesses, $n = 5$, $p = 3$, nothing censored, the same vectors returned on both sides every time.

Table 11: The front-enumeration margin against the size of the front, forty-five instances.

$\|F\|$	instances	ratio (median)	IQR	max
1–4	26	2.7×	[1.7, 3.9]	9.8×
5–12	17	7.5×	[5.2, 16.0]	23.8×
13+	2	31.4×	[24.9, 38.0]	38.0×
all	45	4.1×	[2.3, 7.6]	38.0×



The margin roughly triples and then quadruples as the front grows: $|F|$ *is* the number of cuts. The 13+ row rests on two instances and is quoted as such — this family at $n = 5$ tops out near $|F| = 13$. A separately measured instance with $|F| = 30$ reached 204×; that is an observed extreme, and the typical case on this family is the 4.1× of the last row.

One ratio goes the other way. Counting branch & bound nodes, criterion space explores about three and a half times *more* of them and still finishes four times sooner — its nodes carry no binary and no big-M row, so it wins by solving many easy programs instead of few hard ones. The same count in the margin table goes the other way at every p, because there the two methods solve nearly the same number of sub-programs and the decision-space ones are the harder. Both readings say the same thing: the cost lives in the size of a sub-problem.

9 The complete efficient set, for under one doubling

The front enumeration proves its *vector* list complete and says, in the same breath, that the *point* list is not: one point is kept per slice. Closing that gap turns out to be nearly free, for one reason:

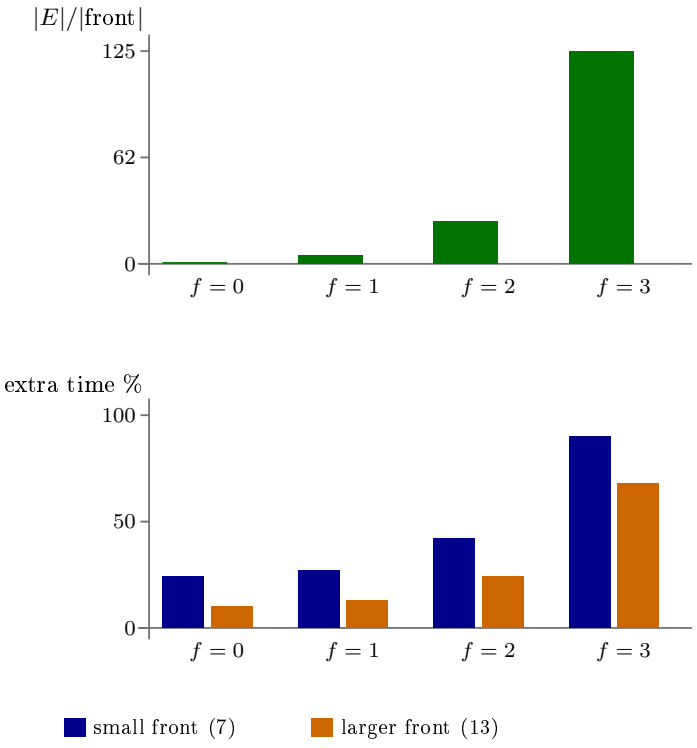
> If v is non-dominated, **every** $x \in D$ with $Z(x) = v$ is efficient — a y dominating such an x would give $Z(y) \geqslant v$ with a strict inequality, making v dominated.

So $E(P_D)$ is exactly the union of the front's slices, and **no efficiency test is paid on any slice point**: the dearest operation in the package is not invoked at all. A slice is D plus p linear equalities — linear even for fractional criteria, since $d^\top x + b > 0$ clears the ratio — so there is no binary, no big-M, and no growth from one slice to the next.

Below, five variables enter Z and f further ones enter only their own bound $0 \leqslant x_j \leqslant 4$: invisible to Z, so each slice is a grid of 5^f points that the front collapses to one.

Table 12: What the slices recover, and what they cost.

f	|front|	$\lvert E(P_D)\rvert$	ratio	slices	extra time
0	7	7	1×	0.016 s	24%
0	13	13	1×	0.018 s	10%
1	7	35	5×	0.019 s	27%
1	13	65	5×	0.024 s	13%
2	7	175	25×	0.036 s	42%
2	13	325	25×	0.050 s	24%
3	7	875	125×	0.103 s	90%
3	13	1625	125×	0.167 s	68%



The two charts are the finding: what is recovered multiplies by five at each step, and the extra time does not. At $f = 3$ the front is *complete and missing* 99.2% *of the points*, and recovering all 1625 of them never doubles the cost. The scan is output-sensitive — about 0.1 ms per point, against 7.68 ms for a single efficiency test, of which none are paid.

The honest reading, which is conditional. At $f = 0$ the front *already was* the efficient set, and the 10–24% buys only the proof of that — the usual case on generic instances, where 476 efficient points sat on 476 distinct vectors over 32 of them. This is not a speed-up and it is not uniform. What it buys is that the method now *knows* which case it is in, rather than returning a point list whose completeness rested on an assumption about the coefficients.

10 The hybridisation that pays, and the three that did not

Nothing above was both *hybrid* and *complete*: $E(P_D)$ was reached by a purely exact method, and every hybridisation had been hung on the optimisation search, where a seed feeds the *bound*. The front enumeration has no bound and no incumbent — it must exhaust every box — so a seed of that kind is worth nothing there.

It has a different mechanism. Every box costs a probe, an efficiency test and sometimes a repair, all spent to find *a vector*; a vector already known needs none of them. Seeding here is not "start from a better value" but "do not pay to discover what you already know".

10.1 First attempt: a generator. It loses.

The augmented Tchebychev program is the natural candidate — its optima are efficient by construction, so they need no verification. Over 120 instances it is *worse the more of the front it seeds*: with a wide weight grid it covers 100% of the front and runs at 2.61× the unseeded time.

The reason is one measurement: a scalarisation is not cheaper than the probe it replaces.

Table 13: One probe against one scalarisation, identical instances.

	time	b&b nodes
one probe	9.48 ms	0
one scalarisation	20.90 ms	21

Cheaper on only 7 of 40. Seeding k vectors saves at most k probes and costs at least k scalarisations plus p for the ideal point — a losing trade before the duplicate weights are counted.

10.2 Second attempt: a walk. It pays — once it is sorted.

A Pareto local search is different in kind: one walk of about 0.011 s yields a whole archive covering 95% of the front. Its points need verification, and that verification is **not** a cost here — the enumeration was going to pay the same efficiency test on every probe the seed removes. It cancels, and the probe is the gain.

But handed over in archive order the hybrid *lost*, at 0.81×, with the probe count *rising* on 4 of 12 instances: the pre-split was building a box structure the search would not have chosen. Sorting the seeds by the direction the probe itself maximises reproduces that structure instead.

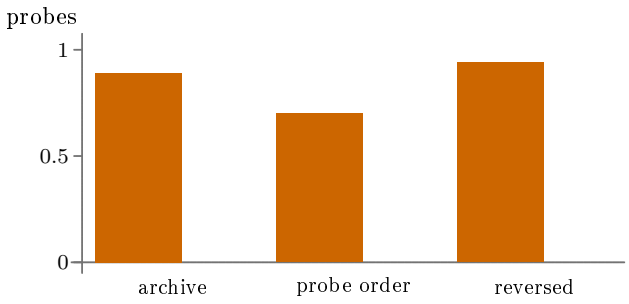


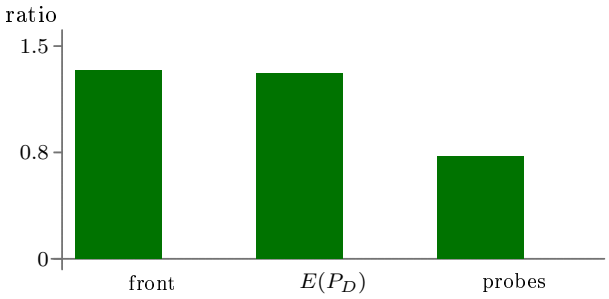
Table 14: Probes against the unseeded search, by the order the seeds arrive.

order	probes	raised the count on
archive order	0.89×	4 of 12
probe order	**0.70**×	**none**
reversed	0.94×	5 of 12

The reversed arm makes the direction causal rather than a coincidence. **A 1.33× win was sitting behind a sort.**

Table 15: The hybrid against the pure exact methods. Thirty instances at each of five configurations, $n = 6 \ldots 9$; medians of per-instance ratios.

	median	IQR	faster on
front enumeration	1.33×	[1.20, 1.51]	139/150
$E(P_D)$ complete	**1.31**×	[1.17, 1.47]	139/150
probes	0.72×	[0.67, 0.75]	*never worse*



Identical answers on all 150 — same front, same $E(P_D)$, same optimum — which the campaign asserts rather than assumes. The heuristic decides only how much of the front has to be discovered.

11 Solving the slice equations: the largest gain here

Completing the front to $E(P_D)$ was dominated by the slice scan: at $n \geqslant 9$ it took 25–30 s against under a second for the front, and at $n = 12$ a slice could not finish inside two minutes.

The p slice equations are *linear*, so solve them. Gaussian elimination in exact rationals expresses r variables as functions of the rest; substituting them into every model row leaves an equivalent system in $n - r$ unknowns. The search space $(ub + 1)^n$ becomes $(ub + 1)^{n-r}$.

Table 16: Pruning on the slice equations against solving them. Fourteen instances, $n = 7 \ldots 12$, the same points every time.

instance	prune	eliminate	ratio
$n = 9$, $p = 3$	29.71 s	0.31 s	96.7×
$n = 10$, $p = 5$	119.47 s	0.92 s	129.9×
$n = 12$, $p = 3$	796.84 s	8.34 s	95.5×
median of all fourteen			**49.2**×

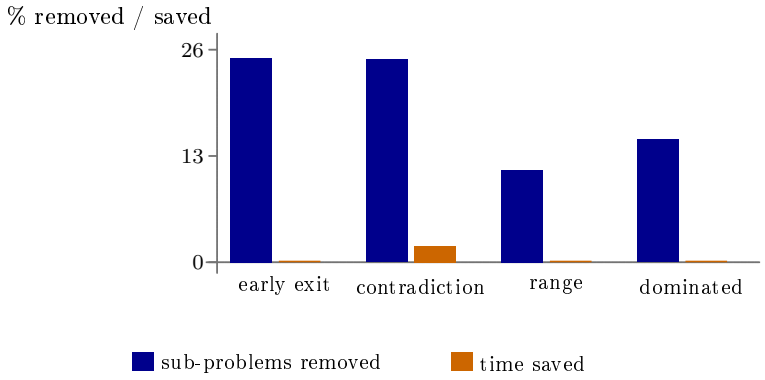
A sign error that ran, and returned nothing. The first draft put a plus where the eliminated variables' contribution must be subtracted. It ran without error, looked like a working optimisation, and returned the *empty* set on 179 of 185 slices — silently dropping efficient points. Only the comparison against the un-eliminated scan caught it.

12 A wall met four times

Four separate changes remove between a fifth and a quarter of the sub-problems and buy nothing.

Table 17: Four attempts on the sub-problem count.

change	sub-problems removed	cost
early exit on a hopeless bound	25%	no measurable time
free contradiction filter	24.8%	0.98× pivots
range filter	+11.3%	1.03× pivots
dominated-whole filter	8–23%	1.077× pivots



Three of the four cost time rather than saving it; a bar cannot be drawn negative, so they sit at zero. The gap between the two series is the finding.

> **In this search, removing sub-problems does not remove time, because the ones that can be removed cheaply are the ones that were already cheap.**

An empty box — and 73% of them are — is refuted by the simplex almost at once: every probe in these families costs *zero* branch & bound nodes, because the relaxation is integral or infeasible at the root. A filter that catches empty boxes catches the cheapest work in the search. The only change that went round this wall attacked an *exponent* rather than a count, and is the 49× above.

The same wall on the proof. After the walk seeds 95% of the front, 93% of the remaining boxes are empty and the enumeration is almost purely a *proof of emptiness*. "Is there a feasible point not dominated by any of the k known ones?" is **one** question, and Sylva–Crema answers it in one integer program built once, with no iteration and no growth. It is sound — 13 of 30 instances proved complete by a single program, no wrong verdict — and 4.5× slower in the median, 100× at worst (239 s against 1.95 s), because k cuts mean kp binaries in one model.

That answers a question wider than the optimisation. Criterion space was claimed to be the better *search* because the model never grows; this tests whether it is also the better *proof*, where nothing grows at all. **It is, by** 4.5×**.**

13 Fractional criteria, where the problem lives

Every figure so far uses *linear* criteria — a gap worth naming in a package about ratios. It has a reason: the Tchebychev comparison needs them, since $z_k^* - Z_k(x)$ is not linear when Z_k is a ratio. The Pareto walk has no such requirement.

Table 18: Genuinely fractional criteria, ten instances per row.

configuration	$\lvert F\rvert$	cover	ratio
$n = 5$, $p = 2$	7	100%	1.54×
$n = 6$, $p = 3$	22	100%	1.35×
$n = 7$, $p = 3$	36	100%	1.25×
all 40			**1.39**×, faster on 34

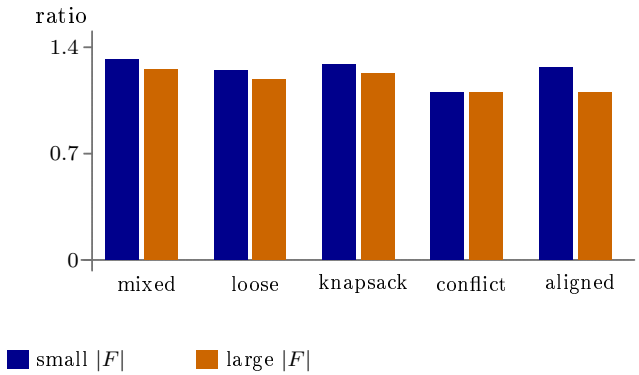
At least as good as the linear case, with the front covered completely. Nothing in the construction distinguishes the two.

14 Five families, and a prediction that was wrong

Everything else here comes from one generator, so the magnitudes could belong to it. Four more families move what the mechanism depends on: `loose` raises $\lvert D\rvert$ an order of magnitude while leaving $\lvert F\rvert$ alone, `knapsack` is two capacity rows with all coefficients positive, `conflicting` gives the criteria opposing signs (large $\lvert F\rvert$), and `aligned` correlates them (small $\lvert F\rvert$).

Table 19: Fifteen instances per cell, 150 in all; two sizes per family. Identical front and identical optimum on every one.

family	$\lvert F\rvert$	cover	ratio	faster on
`mixed`	8 / 30	100 / 97%	**1.37** / 1.30×	28/30
`loose`	9 / 44	83 / 86%	1.29 / 1.23×	28/30
`knapsack`	12 / 41	100 / 90%	1.33 / 1.27×	29/30
`conflicting`	107 / 105	100 / 93%	1.14 / 1.14×	26/30
`aligned`	6 / 11	100 / 84%	1.31 / 1.14×	25/30
all 150			**1.23**×	**136**



Every family pays, 1.14–1.37×. The advantage is not an artefact of the generator it was found with, which is what this was run for.

And the prediction it refutes. We expected `conflicting`, with $|F| \approx 107$, to be the family the hybrid liked *most* — a large front is much to discover, and discovery is what a seed removes — and `aligned` with $|F| = 6$ to be the least. **The opposite holds**, and in the chart the orange bar is below the blue one in four families and level with it in the fifth — never above: the margin narrows as $|F|$ grows, within families as well as across them.

What bounds it, and what does not. A seed removes *one* probe, and the enumeration spends 2.3 to 7.5 probes per vector — the rest go to boxes that turn out empty, which no seed can touch. So the *probe-count* saving is capped at one over that figure, and shrinks as it grows: sweeping p at $n = 7$ gives probes-per-vector 2.3, 3.0, 4.0, 4.9, 6.4, 7.5 and probes kept 0.68, 0.75, 0.82, 0.87 — 32% removed at $p = 3$ down to 13% at $p = 6$, over 72 instances with 68 faster.

> That bound is exact and it is about *counts*. **It does not bound the time**, because a seed removes a probe on a non-empty box — which carries an efficiency test behind it — while the probes left behind are mostly on empty boxes and are refuted at once. At $p = 6$ the time ratio exceeds 1/(probes kept). An earlier draft here called it "a hard ceiling of roughly 1.4×"; that is withdrawn.

What does grow with $|F|$ is the walk's cost, one efficiency test per archive member — so a large front buys no more and costs more.

15 Where the hybrid fails, and the line that fixes it

At $n = 12$ the margin falls to 1.00×, and at $n = 12$, $p = 5$ to 0.97×. Our first explanation was that the probe count outgrows $|F|$. **The measurement refutes it**: probes per vector are flat across the range — 2.4, 2.7, 2.7, 2.8, 2.8, 3.0, 3.9, 5.2. What collapses is the share of the front the walk finds, and the speed-up tracks it.

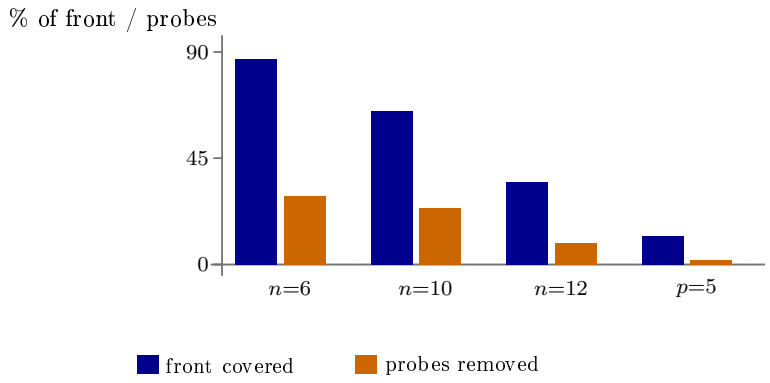


Table 20: Widening the walk alone, on the two instances that failed.

	$n = 12, p = 3$		$n = 12, p = 5$	
restarts / budget	cover	ratio	cover	ratio
8 / 4000	35%	1.03×	12%	0.98×
16 / 20000	95%	**1.61**×	46%	**1.20**×
32 / 60000	95%	1.53×	93%	1.12×
64 / 150000	95%	1.44×	89%	1.17×

Restarts are the knob, not neighbours, and the fix is one line: the first round is sized to the instance. *Not* a second constant — 4000 was chosen at $n = 7, 8$ and generalised, which is what produced the collapse.

But coverage is not the objective. The $p = 5$ column says so directly: 46% **coverage gives** 1.20× **and** 93% **gives** 1.12×. A walk that restarts until it finds nothing new takes coverage there from 45% to 97% and is *slower* — 1.02× against 1.14×, and a median 1.32× against 1.36× over 120 instances. It is in the code and switched off.

A correction to our own reading. The collapse was first reported as "35% coverage, 1.00×" from *one* instance; over six seeds of that configuration the unmodified walk averages 78% and 1.35×. Real and severe where $|F|$ is large, but not as general as one instance made it look — the fifth time here that a single instance misled.

16 How the campaign was measured

Most figures here were first taken on four to eight instances. The *direction* of each was argued from a mechanism, which is what made them worth reporting — but a total over four runs is not an estimate. `studies/campaign.py` re-measures five of the claims over thirty instances per configuration — the first four of them in 751 seconds — and writes every per-instance record to a JSON file so the tables can be rebuilt and re-checked without paying for the run again.

- **The statistic is the median of the per-instance ratios**, with the interquartile range, never a ratio of totals. A ratio of totals is decided by the single heaviest instance — which is exactly how the 103× at $p = 5$ came about.
- **Deterministic work is counted next to the seconds**: sub-programs solved and branch & bound nodes explored, identical on every run and every machine. This machine's noise floor is roughly ±30% per instance, so seconds alone cannot resolve a small effect and no number of repeats fixes that.
- **Censoring is reported, never hidden.** A run that spends its time budget is excluded from the medians and counted in the open, and a row with more than half its runs censored says so instead of quoting a number. In the margin experiment censoring is one-sided — it can only flatter the decision-space method — so those margins are conservative.
- **Agreement is asserted, not assumed.** Every experiment checks that the two methods return the same answer, not merely that one is faster.

Nothing else ran on the machine during the campaign; a contaminated measurement is discarded rather than quoted, which is a rule adopted here only after breaking it once.

17 Every verdict, including the losses

Nine are losses and one is a tie. They are reported at the same length as the wins because they carry the same information: a change pays only where the method has a mechanism it can feed, that mechanism is cheaper to identify than the change is to build and measure, and two ideas feeding the same mechanism are one idea counted twice.

18 How all of this is checked

Exact rational arithmetic throughout, no floating point anywhere, no third-party dependency, and 106 tests run on every commit. Several check *properties* rather than answers, since a method that silently lost efficient points would still agree with the optimum on favourable instances: the split is a partition; the metaheuristic never hands over an inefficient point; every remembered point is efficient; a warm root reaches the same value as the cold one it replaces; the early exit leaves boxes open and still proves optimality.

Table 21: The twenty-two things tried, with what each was worth.

attempt	verdict	effect
decision space → criterion space	**win**	1.5× at $p = 2$ to 14.6× at $p = 6$
exact–exact handover ($s = 1$)	tie	8.47 s vs 8.11 s
exact–exact handover ($s = 3$)	**loss**	13.14 s vs 8.47 s
exact–metaheuristic (Pareto archive)	**win**	1.10× median, 24/30 seeds exact
exact–Tchebychev seed	**loss**	1.02×: a wash, never pays
Tchebychev as a generator	**win**	reaches unsupported points
incumbent seeding in decision space	**loss**	zero iterations saved
LP pre-filter on each box	**loss**	7.95 → 8.36 s, box counts identical
two-at-a-time split	**loss**	loses across $p = 3..7$, worse as p grows
early exit on a hopeless bound	partial	25% fewer boxes, no measurable time
early exit *on top of* the seed	**loss**	4% rather than 25%: a substitute
warm start + remembered point	**win**	20% fewer pivots together
zero-objective probe	**loss**	4–13× slower, cuts identical
slices instead of an efficiency test	**win**	all of $E(P_D)$, under 2×
eliminating the slice variables	**win**	49× median, up to 130×
Tchebychev seeding the enumeration	**loss**	worse the more it seeds
Pareto walk seeding it	**win**	**1.31**× to all of $E(P_D)$
free box filters (three of them)	partial	2%, then two losses
one-program completeness certificate	**loss**	4.5× slower, sound
sizing the walk to the instance	**win**	1.00 → 1.53× at $n = 12$
a walk that restarts until it stops	**loss**	more coverage, less speed
the hybrid on fractional criteria	**win**	1.39×, better than linear

Answers are cross-checked three ways: against the decision-space method, against exhaustive enumeration on small instances, and against a Φ-ordered scan on instances up to 34635 feasible points.

18.1 The references were audited, and nothing here moved

Every margin in this document is a ratio against one of those three reference methods, so the margins are worth exactly what the references are. We audited them and found eleven places in the implementation where a stated hypothesis was never tested. All eleven are fixed and carry regression tests. Two were in the validation apparatus itself and so bear directly on the numbers above.

The completeness certificate searches for challengers inside a variable box it is handed. A box smaller than D hides them, and the certificate then returned *valid* on a point that is not optimal — with the box $[1, 1]$ it certified a point at $\Phi = 1/9$ while the true optimum is $5/17$. Worse, the same defect sat in the two reference oracles. Handed a box smaller than D neither failed nor warned: they answered confidently about the efficient set of a *different* feasible set, under the names $E(P_D)$ and "the exact optimum of (P_E)". On $x_1 + x_2 \leqslant 6$, $x_1 - x_2 \leqslant 2$, whose efficient set holds five points and whose optimum is $2/5$, the box $[1, 1]$ gave one point reported as $E(P_D)$ and $1/3$ reported as the exact optimum. A clipped oracle does not produce a wrong answer; it produces a wrong *standard*, and a method measured against it looks correct precisely when it agrees with the clipping.

Three observations establish that no number in this document was affected, and we state them as observations because the point of the audit is not to assume such things. First, every figure here was produced by passing the box the model derives for itself, never a hand-written one. Second, the oracles now refuse a box that does not enclose D, and with that check active the suite passes

103/103 and the four examples that use the oracles all run clean — which is the evidence that none of the roughly 45 boxes written by hand across the tests and examples was ever clipping D. Third, the scaling study still certifies every instance from $n = 10$ to $n = 20$ after the certificate was guarded, so the guard did not invalidate the study that rests on it. The margins stand as reported: 1.36× to the complete efficient set, with all eleven guards active.

The guards are permanent rather than a debug mode, because they are cheap: boundedness is one integer program for the whole problem, the denominators one per genuinely fractional criterion and none for linear ones, and the variable box n programs cached on the problem. Against the sub-problem counts of this document they are invisible.